Movie Booking Management System

Complete Step-by-Step Code Execution & Codebase Architecture Guide

This engineering handbook provides a complete walkthrough of the CineBook platform. It explains **exactly how to configure and run the backend and frontend**, details the **underlying execution lifecycle**, and walks step-by-step through every database model, controller, service, middleware, and user interface component.

System Core: Fastify 5 (Node.js/TS)	**Database:** PostgreSQL 16 (Sequelize ORM)	**Frontend:** React 19 + TanStack Router
Auth Model: Stateless JWT (@fastify/jwt)	**Concurrency:** Row Locks (SELECT FOR UPDATE)	**Styling:** TailwindCSS + Glassmorphism

1. Step-by-Step Guide: How to Run the System

Follow these structured steps in sequence to boot the PostgreSQL database, seed initial data, launch the Fastify backend API, and start the React client.

Step 1.1: Verify System Prerequisites

Ensure Node.js (v18 or higher), npm (v9+), and PostgreSQL (v14+) are installed and accessible in your system terminal path.

POWERSHELL / BASH

```
node -v # Recommended: v18.0.0 - v24.x
npm -v # Recommended: v9.x - v11.x
psql --version # PostgreSQL 14, 15, or 16
```

Step 1.2: PostgreSQL Database Creation & Config

Open your PostgreSQL CLI (psql) or pgAdmin tool and create the target database named **movie_booking**:

SQL / PSQL CLI

```
CREATE DATABASE movie_booking;
\c movie_booking;
-- Verify database is ready for table synchronization
```

Review database credentials in **backend/src/config/database.ts** to ensure host, port, username, and password match your local environment:

TYPESCRIPT (DATABASE CONFIG)

```
// File: backend/src/config/database.ts
import { Sequelize } from "sequelize";

const sequelize = new Sequelize("movie_booking", "postgres", "Ananth@24", {
    host: "localhost",
    port: 5432,
    dialect: "postgres",
    logging: false, // Set to console.log for SQL debug queries
});
export default sequelize;
```

Step 1.3: Backend Initialization & Startup Flow

Navigate to the backend folder, install dependencies, seed the initial dataset, and launch the dev server:

POWERSHELL / TERMINAL 1

```
# 1. Navigate to backend directory
cd backend

# 2. Install backend dependencies (Fastify, Sequelize, pg, @fastify/jwt, @fastify/cors)
npm install

# 3. Seed verified database records (Users, Theaters, Movies, Shows, Seat Maps)
npm run seed

# 4. Launch backend development server on Port 5050
npm run dev
```

AUTOMATED SEEDING LIFECYCLE: What happens during npm run seed?

1. Authenticates database connection with PostgreSQL.
2. Executes sequelize.sync({ alter: true }) to create tables with foreign key constraints.
3. Inserts default Admin (admin@cinebook.com) and User (user@cinebook.com).
4. Creates 3 Theaters with configured seat capacities (60, 80, 40 seats).
5. Adds 4 blockbuster movies (Oppenheimer, Interstellar, Dune 2, The Dark Knight).
6. Schedules upcoming showtimes and calls ensureSeatsForShow() to generate all numbered seats.

Step 1.4: Frontend Client Installation & Startup

Open a **second terminal window**, navigate to the frontend folder, and boot the Vite development server:

POWERSHELL / TERMINAL 2

```
# 1. Navigate to frontend directory in a separate terminal
cd frontend

# 2. Install React 19, TanStack Router, TailwindCSS, Axios, Lucide Icons
npm install

# 3. Start Vite Development Server with Hot Module Replacement (HMR)
npm run dev

# Output: VITE v8.x ready in ~200 ms
# Local: http://localhost:5173/
```

Step 1.5: Pre-Configured Credentials & Health Check

Role	Email	Password	Access & Permissions
Admin	`admin@cinebook.com`	`Admin@123`	Full Control: Dashboard analytics, manage movies, theaters, shows, and all bookings.
Customer	`user@cinebook.com`	`User@123`	Client Portal: Browse catalog, interactive seat selection, e-tickets, self-cancellation.

VERIFICATION CHECK: API Health Verification: Open your browser or curl: http://localhost:5050/api/health. It returns {"ok": true, "service": "cinebook-backend"} confirming active API communication.

2. Full Codebase Architecture & File Hierarchy

CineBook is organized into strict architectural boundaries ensuring clean separation of concerns, high maintainability, and end-to-end TypeScript type safety.

Architectural Layer	Core Technologies	Responsibilities & Flow
Presentation Layer **Client (SPA)**	React 19, TypeScript, TanStack Router, TailwindCSS, Lucide Icons, Axios	Renders responsive UI, captures user actions, performs client-side validation, handles JWT persistence in localStorage, and manages seat selection state.
API & Routing Layer **Server / Gateway**	Fastify 5, @fastify/cors, @fastify/jwt	High-throughput HTTP request dispatching, CORS headers enforcement, route prefixing, and payload parsing with low-overhead JSON serialization.
Security & Middleware **Auth Guards**	JWT Verification, Role-Based Guards	Validates Bearer tokens on protected endpoints (`authenticate`) and restricts administrative endpoints to users with `role === 'admin'` (`adminOnly`).
Controllers & Services **Business Logic**	TypeScript Controllers, seatService.ts	Orchestrates ACID transactions, enforces PostgreSQL row-level locking (`SELECT ... FOR UPDATE`), computes seat maps, and calculates revenue metrics.
ORM & Database Layer **Data Persistence**	Sequelize 6, PostgreSQL 16, Connection Pooling	Executes parameterized SQL queries, enforces foreign keys, cascades on delete, tracks timestamps (`createdAt`, `updatedAt`), and handles MVCC isolation.

Directory Tree & Source Code Mapping

SYSTEM DIRECTORY TREE

```
MovieBookingManagementSystem/
■■■ backend/
■ ■■■ src/
■ ■ ■■■ config/database.ts # Sequelize PostgreSQL connection & pool config
■ ■ ■■■ models/ # Relational database schema definitions
■ ■ ■ ■■■ User.ts # User identity & roles ('admin' | 'user')
■ ■ ■ ■■■ Movie.ts # Movie catalog details, duration & poster URLs
■ ■ ■ ■■■ Theater.ts # Venues, locations, and total seat capacity
■ ■ ■ ■■■ Show.ts # Movie screenings linked to Theaters & Dates
■ ■ ■ ■■■ Seat.ts # Show seat status ('available' | 'booked')
■ ■ ■ ■■■ Booking.ts # Ticket reservations with confirm/cancel states
■ ■ ■ ■■■ associations.ts # Model foreign keys and relationship joins
■ ■ ■■■ middleware/
■ ■ ■ ■■■ authMiddleware.ts # JWT validation and Admin role guard
■ ■ ■■■ services/
■ ■ ■ ■■■ seatService.ts # Dynamic seat generation & self-repair engine
■ ■ ■■■ controllers/ # Request handling & transaction coordination
■ ■ ■ ■■■ authController.ts # User registration & JWT authentication
■ ■ ■ ■■■ bookingController.ts # Row-locking checkout, cancellation & metrics
■ ■ ■ ■■■ movieController.ts # Movie CRUD operations
■ ■ ■ ■■■ theaterController.ts # Theater CRUD operations
■ ■ ■ ■■■ showController.ts # Show scheduling & seat trigger
■ ■ ■ ■■■ seatController.ts # Seat layout query & repair routes
■ ■ ■ ■■■ adminController.ts # Dashboard metric aggregations
■ ■ ■■■ routes/ # Fastify endpoint prefix registrations
■ ■ ■■■ server.ts # Fastify app instantiation & DB startup
■ ■ ■■■ seed.ts # Automated database seeder script
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ components/Navbar.tsx # Adaptive header for Guest, User, and Admin
■ ■ ■■■ pages/ # React page views & interaction workflows
■ ■ ■ ■■■ Home.tsx # Hero showcase & trending movies grid
■ ■ ■ ■■■ Movies.tsx # Live search, genre filtering & sorting
■ ■ ■ ■■■ MovieDetails.tsx # Movie synopsis & upcoming showtime selector
■ ■ ■ ■■■ SeatSelection.tsx # Interactive SVG curved cinema seat map
■ ■ ■ ■■■ BookingConfirmation.tsx# Printable digital tear-away e-ticket
■ ■ ■ ■■■ MyBookings.tsx # User booking history & instant self-cancellation
■ ■ ■ ■■■ AdminDashboard.tsx # Live metric cards & revenue aggregation
■ ■ ■ ■■■ AdminMovies.tsx # Movie creation, editing & removal modal
■ ■ ■ ■■■ AdminTheaters.tsx # Venue capacity management
■ ■ ■ ■■■ AdminShows.tsx # Show scheduling & seat regeneration
■ ■ ■ ■■■ AdminBookings.tsx # Global booking search, filters & admin overrides
■ ■ ■■■ router.tsx # TanStack Router type-safe route tree
■ ■ ■■■ main.tsx # Root DOM mounting & provider bootstrap
```

3. Detailed Step-by-Step Codebase Walkthrough

This section deconstructs every key file in the codebase, explaining the logic, architecture, algorithms, and design choices.

3.1 Core Server Engine (`backend/src/server.ts`)

The server file initializes the Fastify application, registers essential plugins, establishes the PostgreSQL connection, and mounts modular REST routes.

TYPESCRIPT (server.ts EXCERPT)

```
// 1. Initialize Fastify with structured logging
const app = Fastify({ logger: true });

// 2. Register Cross-Origin Resource Sharing (CORS) to allow React client communication
app.register(cors, {
    origin: true,
    methods: ["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
    allowedHeaders: ["Content-Type", "Authorization"],
});

// 3. Register JWT Plugin for stateless token authentication
app.register(jwt, { secret: "movie-booking-secret-key-change-this-later" });

// 4. Register Modular Route Prefixes
app.register(authRoutes, { prefix: "/api/auth" });
app.register(movieRoutes, { prefix: "/api/movies" });
app.register(theaterRoutes, { prefix: "/api/theaters" });
app.register(showRoutes, { prefix: "/api/shows" });
app.register(seatRoutes, { prefix: "/api/seats" });
app.register(bookingRoutes, { prefix: "/api/bookings" });

// 5. Connect to PostgreSQL and Synchronize Schema
const start = async () => {
    await sequelize.authenticate();
    await sequelize.sync({ alter: true }); // Sync models to DB schema
    await app.listen({ port: 5050, host: "0.0.0.0" });
    console.log("■ Server running on http://localhost:5050");
};
```

Key Technical Takeaways:

• **Fastify vs. Express:** Fastify utilizes an advanced asynchronous architecture delivering over 30,000 req/sec with minimal overhead.
• **sequelize.sync({ alter: true }):** Ensures table schemas, indexes, and column types are safely updated without dropping existing operational data.
• **0.0.0.0 Host Binding:** Guarantees the server accepts incoming connections across localhost, Docker containers, and local LAN environments.

3.2 Relational Data Modeling (`models/*.ts` & `associations.ts`)

CineBook enforces strict relational integrity with foreign keys and cascading updates across 6 core entities:

Model	Primary Attributes	Relationships & Integrity Constraints
User	id, name, email (UK), password, role ('user'|'admin')	User.hasMany(Booking) • Foreign key userId on Booking.
Movie	id, title, genre, description, duration, rating, image	Movie.hasMany(Show) • Foreign key movieId on Show.
Theater	id, name, location, totalSeats	Theater.hasMany(Show) • Foreign key theaterId on Show.

Show	id, movieId, theaterId, showTime, price	Show.belongsTo(Movie), Show.belongsTo(Theater), Show.hasMany(Seat), Show.hasMany(Booking).
Seat	id, showId, seatNumber, status ('available'|'booked')	Seat.belongsTo(Show), Seat.hasMany(Booking) • Unique per (showId, seatNumber).
Booking	id, userId, showId, seatId, status ('confirmed'|'cancelled')	Booking.belongsTo(User), Booking.belongsTo(Show), Booking.belongsTo(Seat).

3.3 Dynamic Seat Generation Engine (`services/seatService.ts`)

A common point of failure in cinema applications is missing or ungenerated seat records. CineBook resolves this via **ensureSeatsForShow()**, an idempotent service that synchronizes seat layouts dynamically:

TYPESCRIPT (services/seatService.ts EXCERPT)

```
export async function ensureSeatsForShow(showId: number, transaction?: Transaction) {
    // 1. Fetch Show and target Theater by direct Foreign Key
    const show = await Show.findByPk(showId, { transaction });
    if (!show) throw new Error("SHOW_NOT_FOUND");
    const theater = await Theater.findByPk(show.theaterId, { transaction });

    // 2. Resolve capacity (defaults to theater.totalSeats or 100)
    const capacity = await resolveCapacity(theater, showId, transaction);

    // 3. Find existing seats for this show
    const existingSeats = await Seat.findAll({ where: { showId }, transaction });
    const existingNumbers = new Set(existingSeats.map((s) => String(s.seatNumber)));

    // 4. Compute missing seats and bulk insert with duplicate protection
    const missingSeats = [];
    for (let num = 1; num <= capacity; num++) {
        if (!existingNumbers.has(String(num))) {
            missingSeats.push({ showId, seatNumber: String(num), status: "available" });
        }
    }
    if (missingSeats.length > 0) {
        await Seat.bulkCreate(missingSeats, { transaction, ignoreDuplicates: true });
    }
    return { show, theater, seats: await Seat.findAll({ where: { showId }, transaction }) };
}
```

3.4 Concurrency Control & Row Locking (`controllers/bookingController.ts`)

The centerpiece of CineBook's backend architecture is its **atomic, race-condition-proof booking transaction**. When high-demand tickets release, multiple customers often attempt to book the exact same seat simultaneously. CineBook utilizes **PostgreSQL Row-Level Locking (SELECT ... FOR UPDATE)** inside an explicit database transaction.

TYPESCRIPT (CONCURRENCY LOCKING IN bookingController.ts)

```
export const createBooking = async (request: FastifyRequest, reply: FastifyReply) => {
    let transaction: Transaction | null = null;
    try {
        const { showId, seatIds } = request.body;
        const userId = request.user.id;

        // Step 1: Ensure seat map integrity before locking
        await ensureSeatsForShow(showId);

        // Step 2: BEGIN TRANSACTION with READ COMMITTED Isolation
        transaction = await sequelize.transaction({
            isolationLevel: Transaction.ISOLATION_LEVELS.READ_COMMITTED,
        });

        // Step 3: EXCLUSIVE ROW LOCK (SELECT ... FOR UPDATE)
        // Subsequent concurrent requests for these seat IDs are held by PostgreSQL
        const seats = await Seat.findAll({
            where: { id: seatIds, showId },
            transaction,
            lock: Transaction.LOCK.UPDATE, // Generates SQL 'FOR UPDATE'
            order: [["id", "ASC"]], // Prevents database deadlocks
        });

        // Step 4: Verify seat status. If ANY seat is already booked, ROLLBACK
        const alreadyBooked = seats.filter((s) => s.status === "booked");
        if (alreadyBooked.length > 0) {
            await transaction.rollback();
            return reply.code(409).send({ message: "One or more selected seats are already booked." });
        }

        // Step 5: ATOMIC WRITE - Create Bookings & Mark Seats as 'booked'
        const createdBookings = [];
        for (const seat of seats) {
            const b = await Booking.create({ userId, showId, seatId: seat.id, status: "confirmed" }, {
transaction });
            await seat.update({ status: "booked" }, { transaction });
            createdBookings.push(b);
        }

        // Step 6: COMMIT TRANSACTION
        await transaction.commit();
        return reply.code(201).send({ message: "Booking successful", bookings: createdBookings });
    } catch (error) {
        if (transaction) await transaction.rollback();
        return reply.code(500).send({ message: "Booking transaction failed" });
    }
};
```

DEADLOCK PREVENTION ALGORITHM: Why order: [['id', 'ASC']] is Critical:
When two concurrent transactions lock multiple rows in opposite orders (e.g., Tx1 locks Seat 5 then 6, while Tx2 locks Seat 6 then 5), a **database deadlock** occurs. By sorting seat IDs in ascending order prior to issuing the FOR UPDATE lock, all transactions acquire row locks in identical sequence, eliminating deadlocks entirely.

3.5 Ticket Cancellation & Instant Seat Release (`cancelBooking`)

When a user or administrator cancels a ticket, CineBook executes an atomic status transition from confirmed to cancelled and immediately unlocks the referenced seat from booked back to available within a single transaction:

TYPESCRIPT (CANCELLATION & SEAT RELEASE)

```
export const cancelBooking = async (request: FastifyRequest, reply: FastifyReply) => {
    const transaction = await sequelize.transaction();
    try {
        const booking = await Booking.findByPk(request.params.id, { transaction });
        if (!booking) { await transaction.rollback(); return reply.code(404).send({ message: "Booking not
found" }); }

        // Update Booking status to 'cancelled'
        await booking.update({ status: "cancelled" }, { transaction });

        // Immediately release the seat back to 'available'
        const seat = await Seat.findByPk(booking.seatId, { transaction });
        if (seat) await seat.update({ status: "available" }, { transaction });

        await transaction.commit();
        return reply.send({ message: "Booking cancelled and seat released successfully" });
    } catch (error) {
        await transaction.rollback();
        return reply.code(500).send({ message: "Cancellation failed" });
    }
};
```

4. Frontend Architecture & User Experience

The CineBook frontend is built using **React 19**, **TypeScript**, and **TanStack Router** for declarative, type-safe client-side routing. The interface leverages TailwindCSS for dark-mode aesthetics, curved cinema screen simulations, and responsive layouts.

4.1 Route Hierarchy (`frontend/src/router.tsx`)

All application routes are registered under a root layout containing the global adaptive Navbar:

URL Path	Component View	Access Level	Key User Actions & Features
`/`	**Home**	Public	Hero billboard banner, trending movie carousel, instant search preview.
`/movies`	**Movies**	Public	Movie catalog with live text search, dynamic genre chips, rating & title sorting.
`/movies/$movieId`	**MovieDetails**	Public	High-res poster, synopsis, duration, and grouped showtime cards with venue details.
`/seats/$showId`	**SeatSelection**	User Auth	Curved theater screen simulation, interactive seat grid, multi-seat cart calculation.
`/booking/$bookingId`	**BookingConfirmation**	User Auth	Digital tear-away cinema ticket, QR code visual accent, native window.print().
`/my-bookings`	**MyBookings**	User Auth	Customer booking ledger with status badges and instant one-click self-cancellation.
`/admin`	**AdminDashboard**	Admin Role	Total users, movies, theaters, scheduled shows, gross revenue (■), and quick action tabs.
`/admin/movies`	**AdminMovies**	Admin Role	Create, edit, and delete movie records with runtime validation.
`/admin/theaters`	**AdminTheaters**	Admin Role	Venue setup and total seat capacity configuration.
`/admin/shows`	**AdminShows**	Admin Role	Show scheduling with automated seat map generation and repair controls.
`/admin/bookings`	**AdminBookings**	Admin Role	Global booking search by customer email/movie, status filter, and admin cancellation override.

4.2 Interactive Seat Selection Engine (`SeatSelection.tsx`)

The seat selection page simulates a real multiplex theater experience. Key technical features include:

1. **Dynamic Visual Layout:** Organizes numerical seats (1 to N) into alphabetical rows (Row A, Row B, Row C...).
2. **Real-Time State Mapping:**
   - **Green / Available:** Clickable seat. Clicking toggles seat ID into the local selection array.
   - **Violet / Selected:** Highlighted seat in current cart. Dynamically computes total price (`count * show.price`).
   - **Red / Booked:** Disabled seat with cursor-not-allowed, preventing selection of already reserved seats.
3. **Multi-Seat Booking Submission:** Dispatches POST /api/bookings with payload { showId, seatIds: [...] }.
4. **Conflict Handling:** If another user finishes checkout first, the UI catches HTTP 409 and prompts the user to refresh the map.

5. Complete REST API Master Reference

The CineBook backend provides 17 RESTful endpoints formatted in JSON, secured with JWT Bearer authentication headers.

Method & Route	Auth / Access	Request Payload	Description & Status Codes
`POST /api/auth/register`	Public	{ name, email, password }	Registers new customer account. Returns 201 Created (409 on duplicate email).
`POST /api/auth/login`	Public	{ email, password }	Validates credentials & returns signed JWT token. Returns 200 OK (401 on invalid).
`GET /api/movies`	Public	None	Retrieves all movies with active show associations. Returns 200 OK.
`GET /api/movies/:id`	Public	None	Retrieves movie details and nested upcoming showtimes. Returns 200 OK (404 if missing).
`POST /api/movies`	**Admin (JWT)**	{ title, genre, description, duration, rating, image }	Creates new movie record. Returns 201 Created.
`PUT /api/movies/:id`	**Admin (JWT)**	{ title, genre, duration, ... }	Updates existing movie attributes. Returns 200 OK.
`DELETE /api/movies/:id`	**Admin (JWT)**	None	Deletes movie record. Returns 200 OK.
`GET /api/theaters`	Public	None	Retrieves all theaters with total seat capacities. Returns 200 OK.
`POST /api/theaters`	**Admin (JWT)**	{ name, location, totalSeats }	Creates theater venue with configured seat capacity. Returns 201 Created.
`GET /api/shows`	Public	None	Retrieves all scheduled shows with theater and movie relations. Returns 200 OK.
`POST /api/shows`	**Admin (JWT)**	{ movieId, theaterId, showTime, price }	Schedules show and automatically invokes ensureSeatsForShow(). Returns 201 Created.
`GET /api/seats/show/:showId`	User (JWT)	None	Returns real-time seat status layout (available vs booked). Returns 200 OK.
`POST /api/bookings`	User (JWT)	{ showId, seatIds: [...] }	Atomic multi-seat booking with PostgreSQL row lock. Returns 201 (409 on conflict).
`GET /api/bookings/my`	User (JWT)	None	Returns authenticated user's complete booking history. Returns 200 OK.
`GET /api/bookings/:id`	User (JWT)	None	Retrieves digital e-ticket verification data for printable ticket. Returns 200 OK.
`DELETE /api/bookings/:id`	User (JWT)	None	Cancels booking and releases seat back to 'available'. Returns 200 OK.
`GET /api/bookings/admin/dashboard`	**Admin (JWT)**	None	Aggregates gross revenue (■), total users, movies, and confirmed counts. Returns 200 OK.

6. Troubleshooting & Common Setup FAQs

Issue / Error Message	Root Cause	Step-by-Step Resolution
ConnectionRefusedError: connect ECONNREFUSED 127.0.0.1:5432	PostgreSQL service is not running on your machine or port 5432 is blocked.	Open Windows Services (services.msc) and start the **postgresql-x64** service, or run pg_ctl -D "C:\Program Files\PostgreSQL\16\data" start.
password authentication failed for user "postgres"	Password in backend/src/config/database.ts does not match your local PostgreSQL password.	Update line 6 of database.ts with your actual PostgreSQL superuser password and re-run npm run dev.
EADDRINUSE: address already in use :::5050	An older instance of Node/Fastify is already occupying Port 5050.	Kill the stale process via PowerShell: Get-Process node | Stop-Process -Force or find PID via netstat -ano | findstr :5050.
Seat Map empty on /seats/:id	Show was created in a legacy database before the automated seat trigger was active.	CineBook automatically triggers self-repair upon route access, or the admin can click **Create/Repair Seats** in /admin/shows.

7. Viva & Coordinator Presentation Q&A; Cheat Sheet

Q1: Why did you choose Fastify over Express.js for the backend?

Answer: Fastify delivers up to 2x higher throughput than Express with minimal overhead, native async/await support, structured JSON schema validation, and low latency serialization.

Q2: How does CineBook prevent double-booking during high-traffic ticket sales?

Answer: We use PostgreSQL Row-Level Locking (SELECT ... FOR UPDATE) inside Sequelize transactions. The first request acquires an exclusive lock on seat rows; concurrent requests wait and receive an immediate 409 Conflict when detecting the updated status.

Q3: How does the dynamic seat generation mechanism work?

Answer: The ensureSeatsForShow() service dynamically computes missing seat numbers up to the theater's capacity and bulk-creates them with ignoreDuplicates: true, ensuring zero missing seat layout bugs.

Q4: How is user authentication and role authorization handled?

Answer: Authentication uses stateless JSON Web Tokens (JWT) signed with user ID, email, and role. The authenticate middleware verifies the token, while adminOnly enforces role === 'admin'.

Q5: What happens when a user cancels a confirmed booking?

Answer: An atomic database transaction updates the booking record to cancelled and immediately flips the associated seat status from booked back to available, making it instantly bookable for others.

5-MINUTE DEMONSTRATION SUMMARY: Summary for Coordinator Demonstration:
Start by demonstrating Admin controls (Dashboard metrics, Movies, Theaters, Shows) with admin@cinebook.com. Then log in as user@cinebook.com, filter movies, select seats on the interactive screen, confirm booking to show the printable e-ticket, and demonstrate transactional integrity by showing the seat locked, then cancelling the booking to show immediate seat recovery.